

김성민

CV Engineer · 3년 4개월 경력

CCTV관제, 주차 제어 등의 프로젝트를 경험하며 현장에 적용 가능한 실시간 영상 분석 솔루션을 개발해왔습니다.

📞 010-4879-7216 ✉ 7tkfkd@naver.com

🎓 명지대 물리학과



3년+

CV Engineer 경력

자기 소개

CCTV 관제, 주차 제어, 건축물 결함 검사 등 3개 도메인에서 영상 입력부터 배포까지 엔드투엔드 경험이 있는 엔지니어입니다.

네이버 부스트캠프 당시 자체 개발 증강 알고리즘을 비롯한 증강, W&B를 활용한 모니터링부터 AI 알고리즘 개발 및 배포 과정을 배웠습니다.

이후 현업에선 모델 개발을 넘어 DeepStream·TensorRT·Docker·Jenkins·Bash 기반 배포 파이프라인을 설계·구축하고,

오탐 검출 알고리즘 개선으로 분석 속도 4ms → 0.9ms,

번호판 인식 알고리즘 개선으로 장당 분석시간 0.2s → 0.05s, 정확도 60% → 96.4% 등 현장에서 검증된 성과를 만들어왔습니다.

모델 개발부터 API·운영·고객 현장 지원까지 소프트웨어 개발 전 주기 경험을 보유하고 있습니다.

2023.11 — 2025.06 · 1년 8개월

플렉시티

AX사업부 대리 · CV Engineer

CCTV 자동 관제 솔루션 개발. 오탐 제거, CI/CD 배포 파이프라인, 시스템 안정화.

2022.03 — 2023.11 · 1년 8개월

뷰메진

CV Team 팀원 · CV Engineer

자율비행 드론 안전진단. 주차 관제 시스템(정부과제), 건축물 결함 검사 솔루션.

2021.08 — 2021.12

부스트캠프 AI Tech

네이버커넥트재단

Detection, Segmentation, OCR, 경량화, Product Serving 전 과정 이수.

기술 스택

AI / VISION

- PyTorch
- OpenCV
- TorchVision
- DeepStream
- ONNX
- TensorRT
- W&B

BACKEND / DEPLOYMENT

- Flask
- FastAPI
- Docker
- Jenkins
- PyArmor
- Bash
- Multi-processing

DATABASE / DATA

- MariaDB
- pandas
- NumPy
- scikit-learn
- DVC

LANGUAGE / TOOLS

- Python
- C++
- Git
- Linux

PROJECT 01

CCTV 자동 관제 솔루션

고객 CCTV 영상에서 화재·연기·위험지역 접근 등 이벤트를 실시간 감지, 알림 발생

2023.11 ~ 2025.07 팀 4명

0.9_{ms}

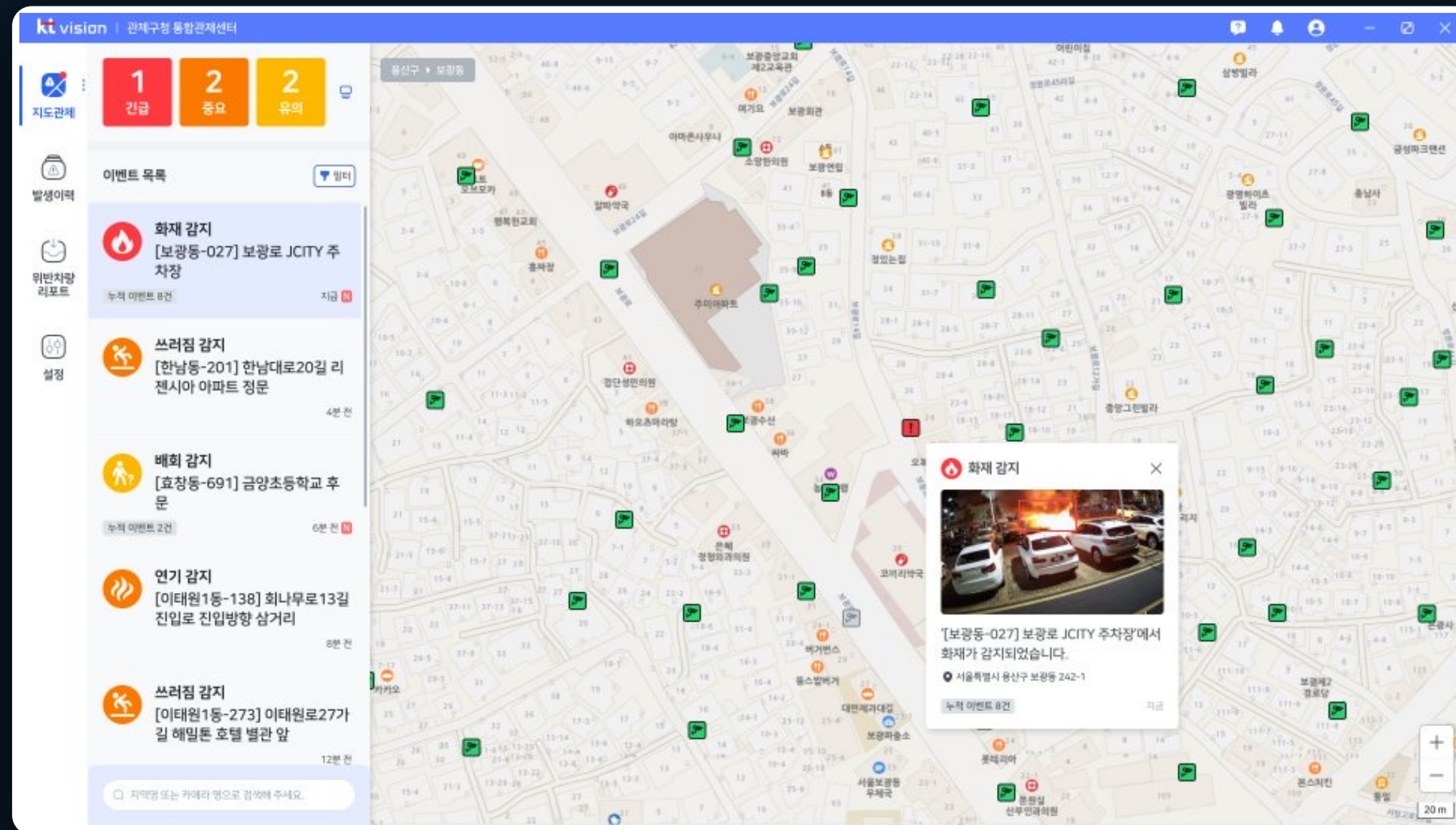
분석속도
(4ms→)

84.3%

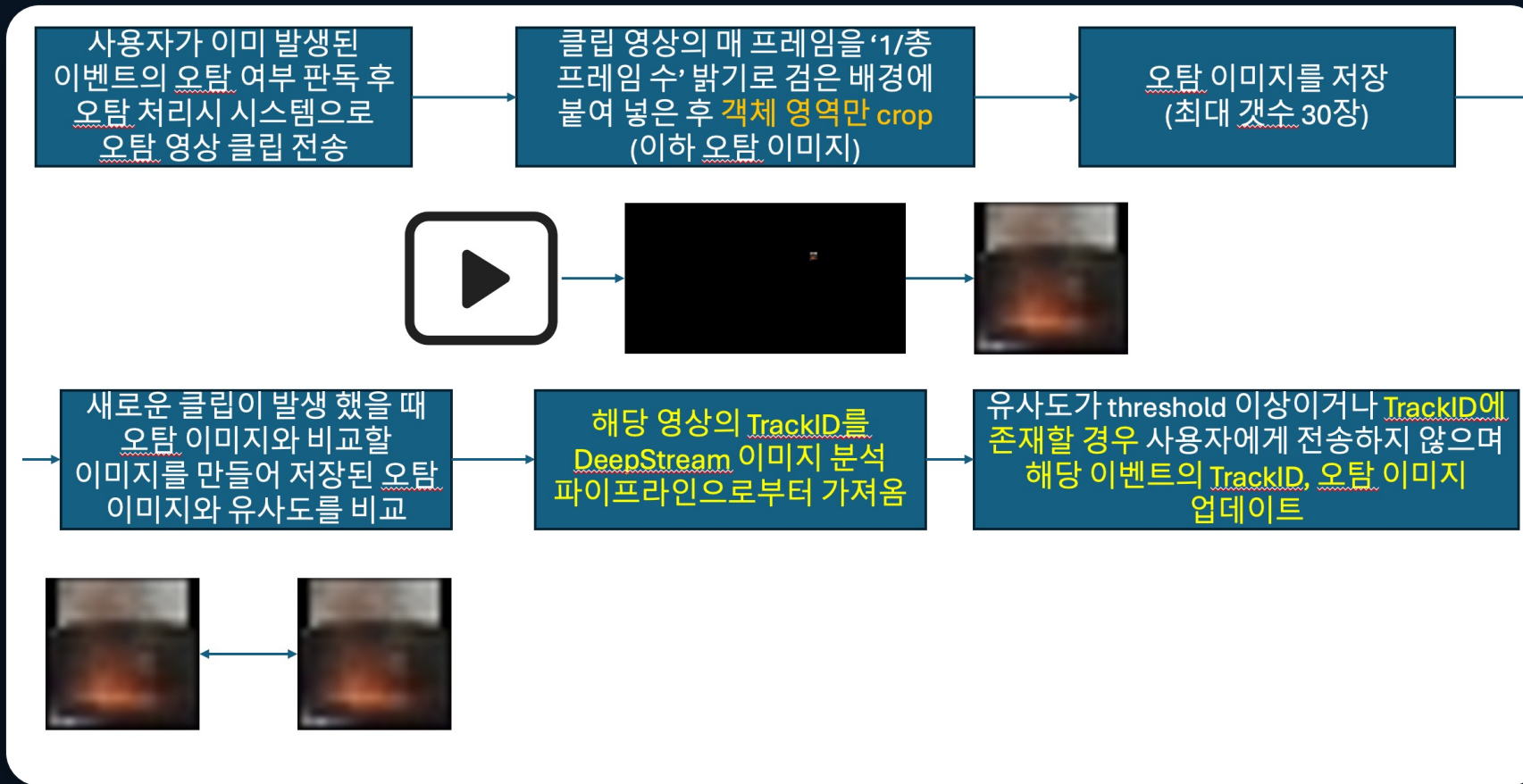
F1-score
(81.9→)

28_{ch}

허용채널
(25→)



오탐 처리 알고리즘 개선



문제 및 해결 과정

문제: 오탐 케이스 처리 한계 + 정확도

기존 오탐 처리 알고리즘은 속도의 문제로 다양한 케이스의 오탐을 처리하지 못하며, 객체가 이동할 경우를 처리하지 못하며 정확도의 개선이 필요한 상황.

해결: Crop + Tracking 알고리즘

객체 영역 Crop 알고리즘을 도입해 정확도, 동작 속도 개선

Tracking 알고리즘 도입을 통해 객체가 이동하는 경우를 해결

성과: F1 84.34, 클립당 분석 0.9ms

F1-score: 81.94 → 84.34

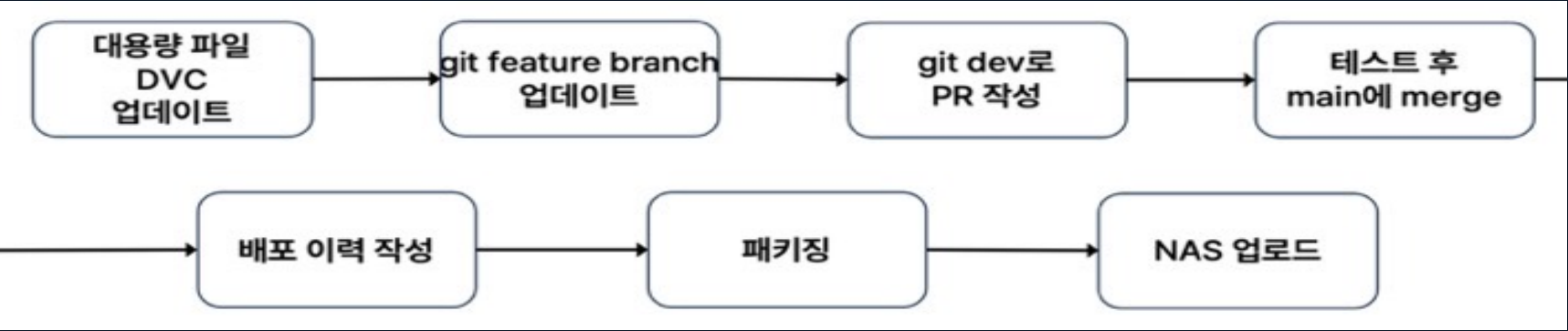
클립당 분석속도 4ms → 0.9ms

💡 AI 사용 포인트

기술 부채 해결

- 스파게티 코드 개선
- SOLID 원칙에 위배된 설계 개선

배포 파이프라인 개선 & 트러블슈팅을 통한 안정화



분석 SW 스펙

담당자 : 김성민 주임

<참고>

- GPU 1개당 Pipeline 1개씩 할당
- GPU Turing arch.이상
- CCTV 채널당 이벤트 1개 등록을 가정한 리소스 사용량

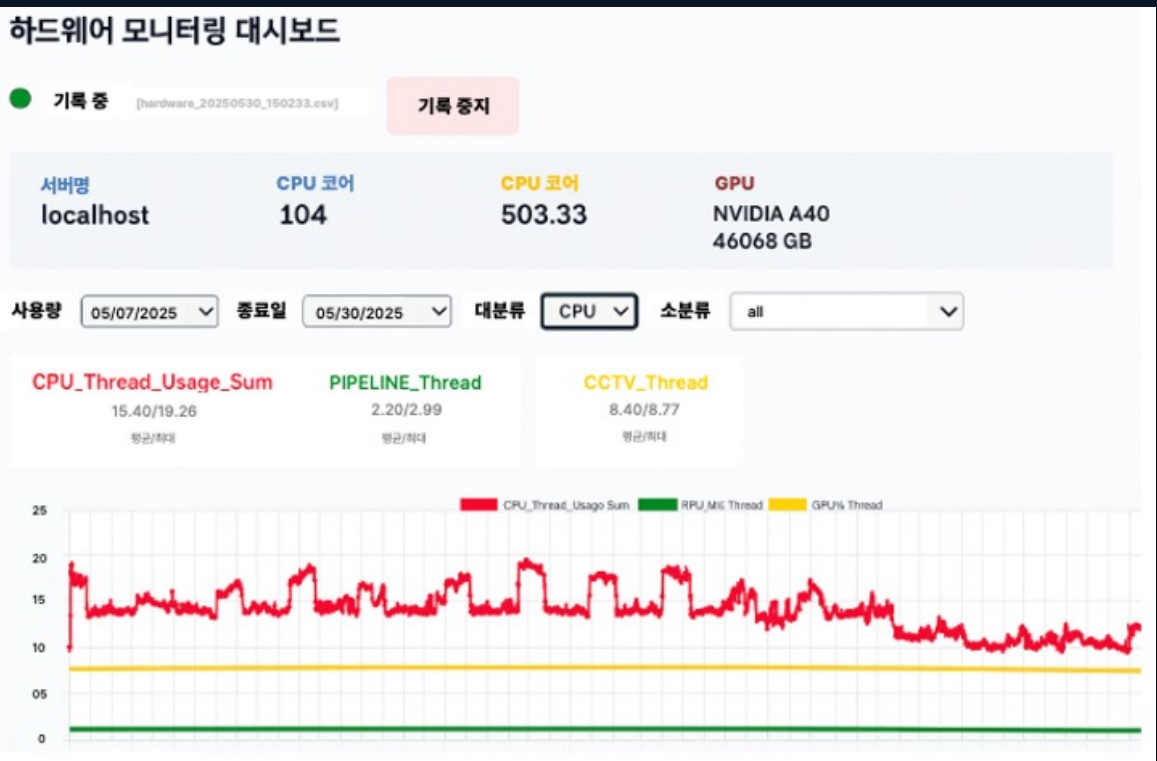
CPU	Pipeline	리소스 사용량 테스트 결과 값
	channel	리소스 사용량 테스트 결과 값
Mem	OS	리소스 사용량 테스트 결과 값
	통신	리소스 사용량 테스트 결과 값
	Pipeline	리소스 사용량 테스트 결과 값
	channel	(‘각 영상별 ‘가로 픽셀 수 * 세로 픽셀 수 * 채널 수 * 저장해두는 프레임 수’의 합) * 2(클립 생성 때도 같은 크기의 메모리를 사용)
	event	리소스 사용량 테스트 결과 값
Storage	OS, Data	저장하려는 데이터의 양에 맞추어 산정
NIC	output	(클립 용량+기타 통신) * 단위 변환: MB > Mbit(*8)
	input	RTSP 영상의 bitrate

<표1> 채널별 단위 값 설정 기준

가로	세로	채널	fps
분석 (input)	1920	1080	3
알람 클립 (output)	960	540	3
공유용	1920	1080	4
frame_diff	1920	1080	1
background	1920	1080	1
opticalflow	480	270	2
ignore	640	360	3

<표2> 사용 영상 별 해상도, fps

하드웨어	단위	사용 항목	단위 값	채널 수에 따른 리소스 사용량															
채널수				25										30					
				2				4				2							
모델개수				A40	A6000	L4	L40	L40S	A40	A6000	L4	L40	L40S	A40	A6000	L4	L40	L40S	A40
GPU																			
		분석 Pipeline	3	3	3	6	3	3	3	3	6	3	3	3	3	3	6	3	3
		CCTV 채널	0.5	12.5	12.5	12.5	12.5	12.5	12.5	12.5	12.5	12.5	12.5	12.5	15	15	15	15	15



문제 및 해결 과정

문제: 불안정한 솔루션, 적은 인력, 폐쇄망 대응력

소수의 인력으로 개발, 디자인, 영업까지 많은 영역을 담당해야 했음.

테스트시에 동작하던 솔루션이 현장에 나가면 동작하지 않는 경우가 존재

해결: RCA, 배포 과정 자동화, SOP를 통한 관리, 테스트 자동화 및 모니터링 개선

배포 과정을 표준화, 패키징 및 자동화한 뒤 설치, 트러블 슈팅 문서, 스펙 문서 등

표준 운영문서를 통해 관리함으로써 인력 문제 해결

모니터링 환경 개선을 통해 스펙 강건화

성과: 점검단위 기간 30일 간 무중단 운영 달성

RCA를 통해 19개 이상의 기존 솔루션의 문제 개선

PyArmor 패키징 알고리즘으로 난독화 및 라이브러리 문제 개선

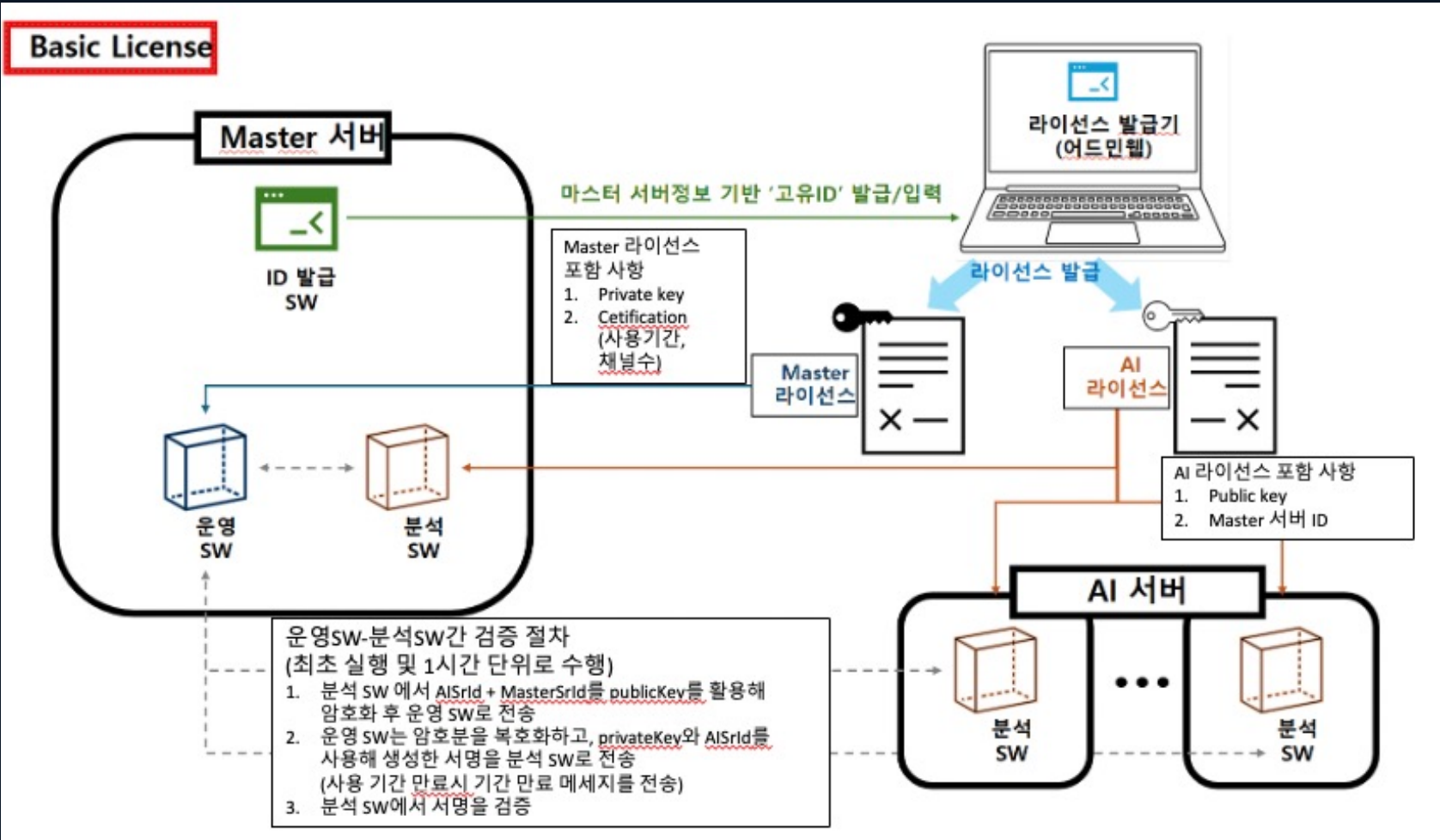
Docker를 활용해 환경 문제 개선

하드웨어 모니터링 툴 및 SOP

스펙화를 통한 파이프라인 개선으로 A40 GPU 기준 채널수 25 → 28

💡 AI 사용 포인트

Cursor를 활용하여 하드웨어 모니터링 툴 구축



문제 및 해결 과정

문제: 인증 체계 부족

시스템상 고객사와 협의된 채널수가 100채널이라 했을 때, 고객사에서 임의로 채널을 추가해도 막을 수 없었음.

또한 외부 업체에서 서버 접근이 가능한 환경이여 SW를 복제할 수 있었음.

해결: License를 활용한 인증 체계 구축

하드웨어 정보, 해싱을 활용한 ID를 활용해 특정 하드웨어에 종속 케이스 정리를 통해 유출 가능 경우의 수 차단

성과: 보안

인증된 서버 환경에서 정해진 채널수만 사용할 수 있는 인증 체계 구축

Backup & Restore 항목 리스트업						
위치	항목	항목 상세	위치 상세	연관 DB 테이블	DB 설명	백업 여부 테스트 방법
Admin Web	카메라 목록	카메라 명, 설치 지역(시/도), 카메라 유형, 위도, 경도, 상세주소, PTZ유형, 카메라 IP, RTSP URL, PTZ 카메라 ID&PW, 열화상 재조사 / 포트번호, 분석서버 이름, 설명	카메라 > 카메라 & 배치 > 카메라 관리	aw_cctv aw_cctv_stat_his	CCTV 목록 CCTV 관련 동작 이력	카메라 목록을 엑셀을 Export 백업 이후 리스토어 된 카메라 목록과 비교
Admin Web	사용자 목록	사용자 아이디, 이름, 휴대폰번호, 권한 등	설정 > 계정 설정 > 사용자	core_user_ext core_user_pass	사용자 목록 및 연락처 사용자 비밀번호	계정 설정에 Test 계정을 만든 후 엑셀을 백업 이후 운영 SW에 Test 계정으로 로딩
Admin Web	서버 목록	서버명, 서버아이디, 해당 서버에 등록된 채널	서버 > 서버 관리 > AI 서버	aw_device aw_device_his	연결된 서버 서버의 동작 이력	서버 관리에 등록된 서버를 확인 백업 이후 등록된 서버가 있는지 확인
Admin Web	지도 축적 Default값	납품 지역별 상이한 지도 축적 사용. 지도 화면 진입 시 초기 축적값	카메라 > 카메라 & 배치 > 지도 배치 관리	없음		지도 축적 값을 변경 한 후 백업 리스토어 후 변경된 지도 축적 값이 보이는지
Admin Web	지도 중앙값	납품 지역별 상이한 중앙값(위경도) 사용. 지도 화면 진입 시, 초기 중앙값 위경도	카메라 > 카메라 & 배치 > 지도 배치 관리	없음		지도 중앙 값을 변경 한 후 백업 리스토어 후 변경된 지도 중앙 값이 보이는지
Admin Web	기관	일반설정에 설정된 기관 명	설정 > 일반 설정 > 기관	auth_menu - ADIWE042300 (백업 불필요) aw_area - area_type = ROOT	헤더를 비롯한 운영 SW에서 관리 하는 메뉴 관제 지역(시/도)	기관을 변경한 후 백업 리스토어 후 변경된 기관 값이 보이는지

문제 및 해결 과정

문제: 백업 기능의 부재

시스템 오류 시 데이터가 유실되는 문제 발생

업데이트 기능 또한 부재하여 시스템 업데이트시 재설치를 하게 되어 기존 히스토리 유실

해결: 백업 기능 구현

백업 해야 하는 항목들을 정리 및 관련 사항들을 표준화

DB 관련 명령어, Bash를 통해 백업 과정 자동화

성과: 백업 기능

시스템 오류, 업데이트 시에도 데이터의 유실 방지

PROJECT 02

주차 관제 솔루션

주차장 입구 차량 입차 확인, 번호판 위변조 탐지, 차번 인식
(NIPA 정부과제)

17 2022.04 ~ 2023.02 팀 7명

96.4%

번호인식
(60%→)

100%

ToF 물체
판별

0.151s

Jetson Nano
번호 인식



번호 인식 알고리즘

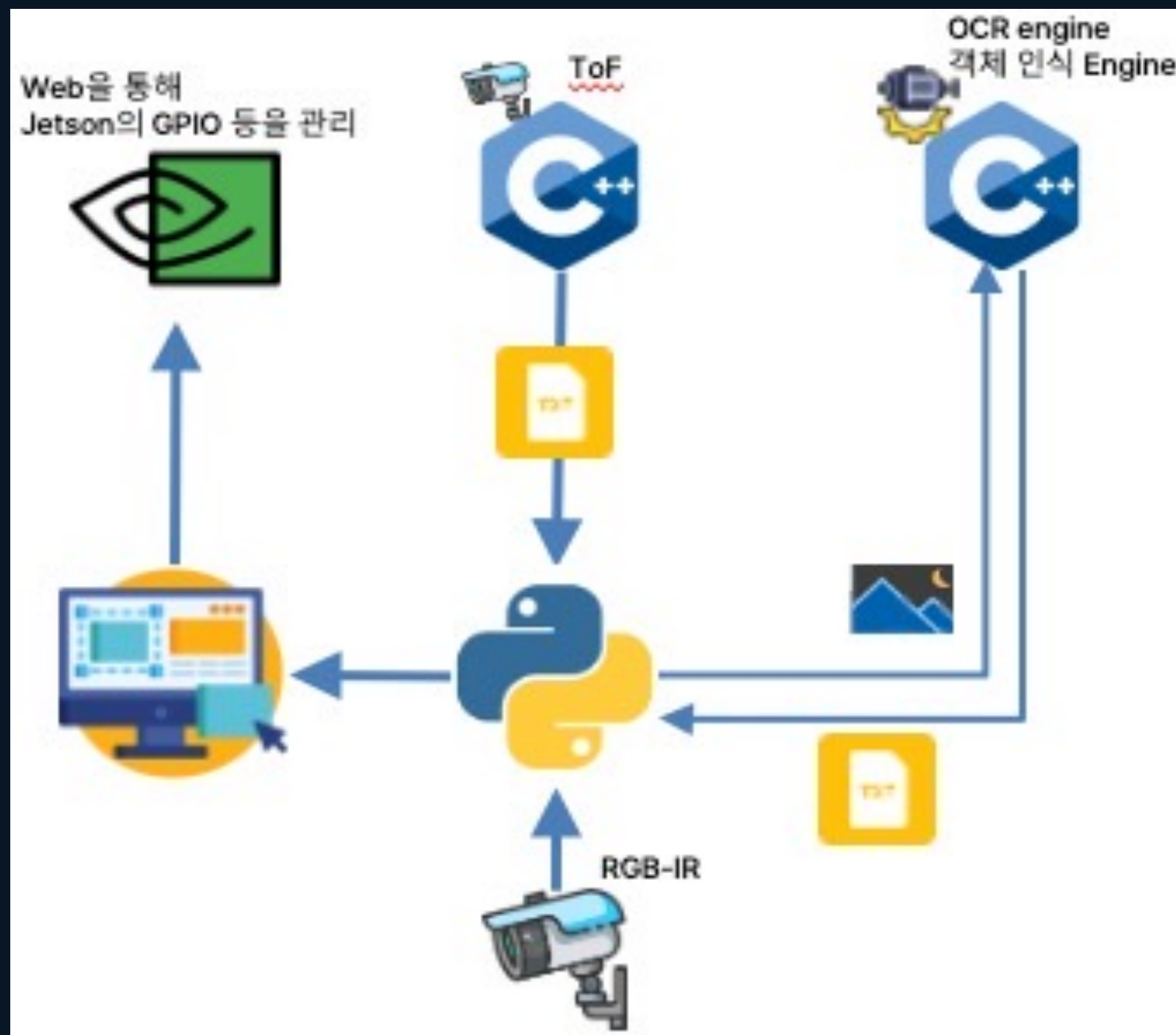


문제 및 해결 과정

문제: 목표 성능에 도달하지 못하는 번호 인식 시스템
과제 목표인 95%를 달성하지 못하고 매번 60% 언저리만 달성된 번호 인식 알고리즘

해결: OCR이 아닌 객체인식 + Voting 알고리즘 도입
Rule 및 객체 인식 알고리즘으로
OCR은 이전 맥락을 유지하는 특징이 있으나 번호판은 맥락이 없는 데이터
이를 대체하고자 객체 인식을 도입
3장의 Frame을 분석하여 Voting 하는 방식으로 정확도를 개선

성과: 과제 목표 달성
Accuracy: 약 60% → 89% → 96.4%



문제 및 해결 과정

문제: ToF, RGB 카메라를 포함하여 Edge에서 동작할 수 있는 시스템

C++ SDK만 지원하는 ToF 카메라, Python 시스템을 통해 테스트된 RGB 카메라

Jetson Nano에서 Realtime 동작 필요

Inference time delay 등의 latency 문제 발생

해결: 경량화 및 파일기반 IPC

TensorRT, ONNX를 통해 경량화 후 C++를 활용해 Inference

버퍼를 포함한 자체 제작 VideoCapture를 사용

C++에서 분석된 결과를 TXT에 기록 Python이 해당 내용을 모니터링하는

성과: Jetson Nano에서 1사이클 당 약 0.25s에 분석 가능한 시스템

차량 객체 인식 0.5s + 번호판 인식 0.5s + 3장의 Voting과 함께 번호 인식 0.151s
= 약 0.25s

PROJECT 03

건축물 결함 검사 솔루션

드론을 활용한 건축물 결함 검사 후 전체 구조물 결함 정보를 담은 리포트 생성

17 2022.03 ~ 2023.11 팀 21명

8K

고해상도
이미지

0.3mm

균열 두께
측정

논문

GPS 균열 위치
알고리즘



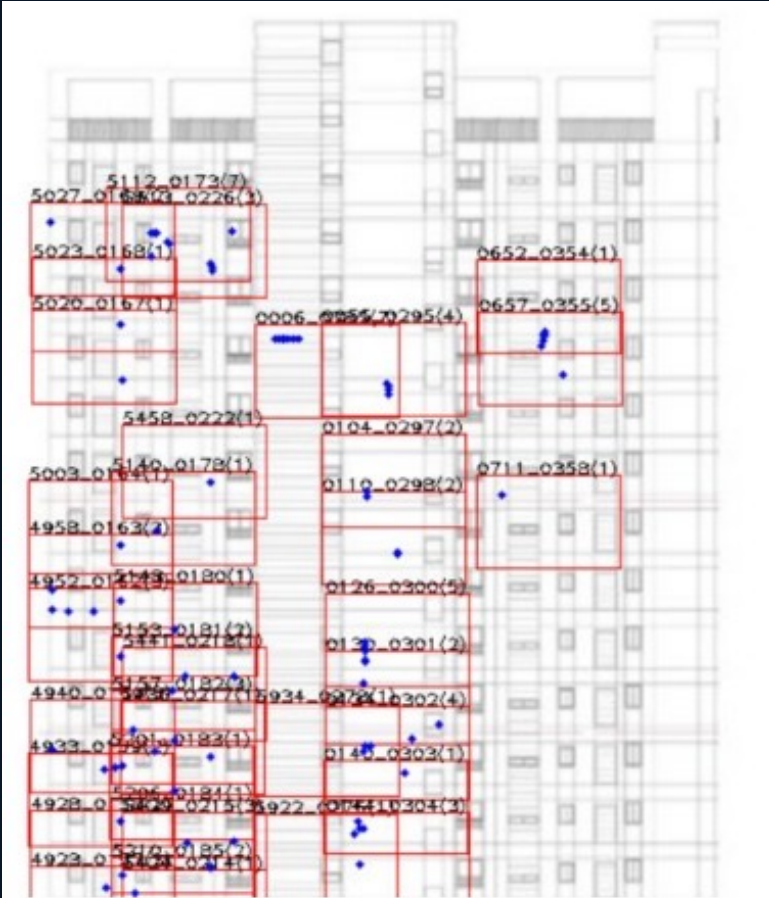


그림 6. 결과물 예시

문제 및 해결 과정

문제: 학습 파이프라인 부재 및 작은 크기의 균열을 찾을 수 있는 알고리즘

초기 스타트업 특성상 학습 파이프라인이 없었음.

0.3mm의 균열을 찾을 수 있어야 하며, 결과물은 건물의 도면 내에 기록될 수 있어야 함.

해결: 초기 학습 파이프라인 설계 및 Loacalization 알고리즘 구현

Dataset → DataLoader → Train(Model Class) → Valid with Metric

으로 이어지는 초기 학습 파이프라인 구축

GPS를 활용한 건축물 내 균열 위치 표기 알고리즘 도입

성과: 94% 정확도 균열 검출 + 균열 위치 표기 알고리즘

초기 학습 파이프라인을 기준으로 점차 Develop 되어 회사 기준 최종 성능 94%를

달성 및 건축물 내 균열 위치 표기 알고리즘 도입 및 논문 작성

교육 & 자격증

2015.03 — 2019.08

명지대학교

물리학과 학사 · GPA 3.49/4.5

경기 본교 자연과학대학. 수학·물리적 사고력을 바탕으로 AI 알고리즘 설계에 강점.

2021.08 — 2021.12

부스트캠프 AI Tech

네이버커넥트재단

딥러닝 기초 · Detection · Segmentation · OCR · 경량화 최적화 · Product Serving

2024.06

ADSP

데이터 분석 준전문가

데이터 분석 기획, 데이터 분석, 데이터 활용 전반에 대한 전문성 인증.

고객의 필요를 해결할 솔루션을 만들 준비가 되어 있습니다

모델 개발뿐 아니라 API 개발, Ops 경험을 바탕으로 현장의 문제를 해결하는
엔지니어

AI / VISION

MLOPS & DEPLOY

END-TO-END 솔루션

PHONE

010-4879-7216

EMAIL

7tkfkd@naver.com

GITHUB

ksm0517